

Laporan Tugas Besar 2

IF2010 Pemrograman Berorientasi Objek 2025/2026

Banana Republic

Kode Kelompok : LLM

Nama Kelompok : Liro Language Model

1. 13524122 / Nathaniel Christian
2. 13524133 / M. Daffa Arrizki Yanma
3. 13524135 / Varistha Devi
4. 13524139 / Azri Arzaq Pohan
5. 13524142 / Rasyad Satyatma

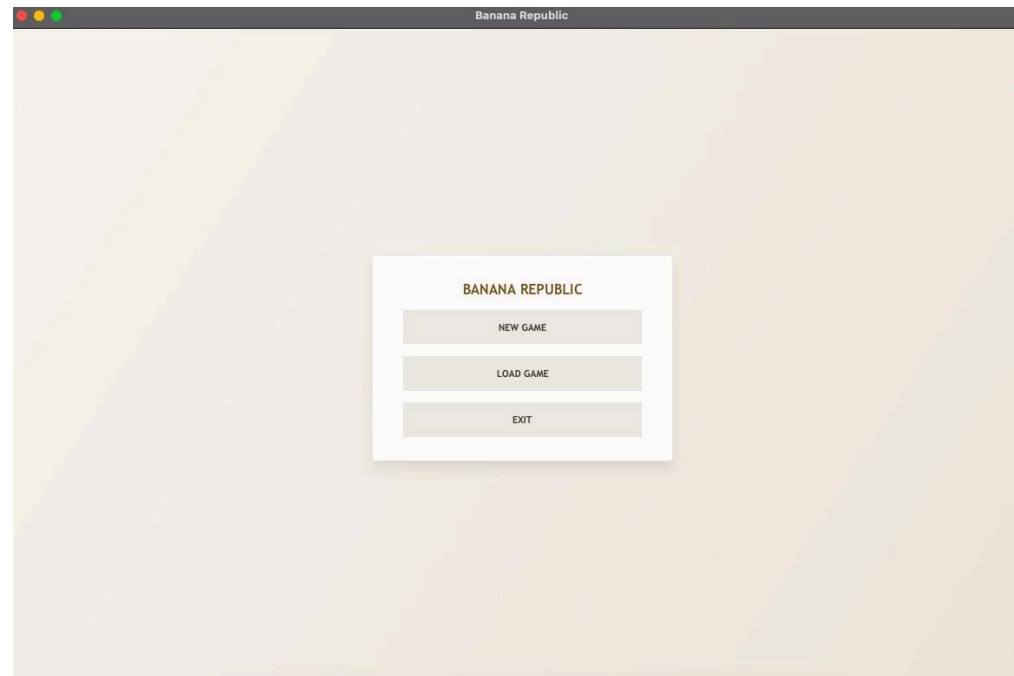


LLM - Liro Language Model

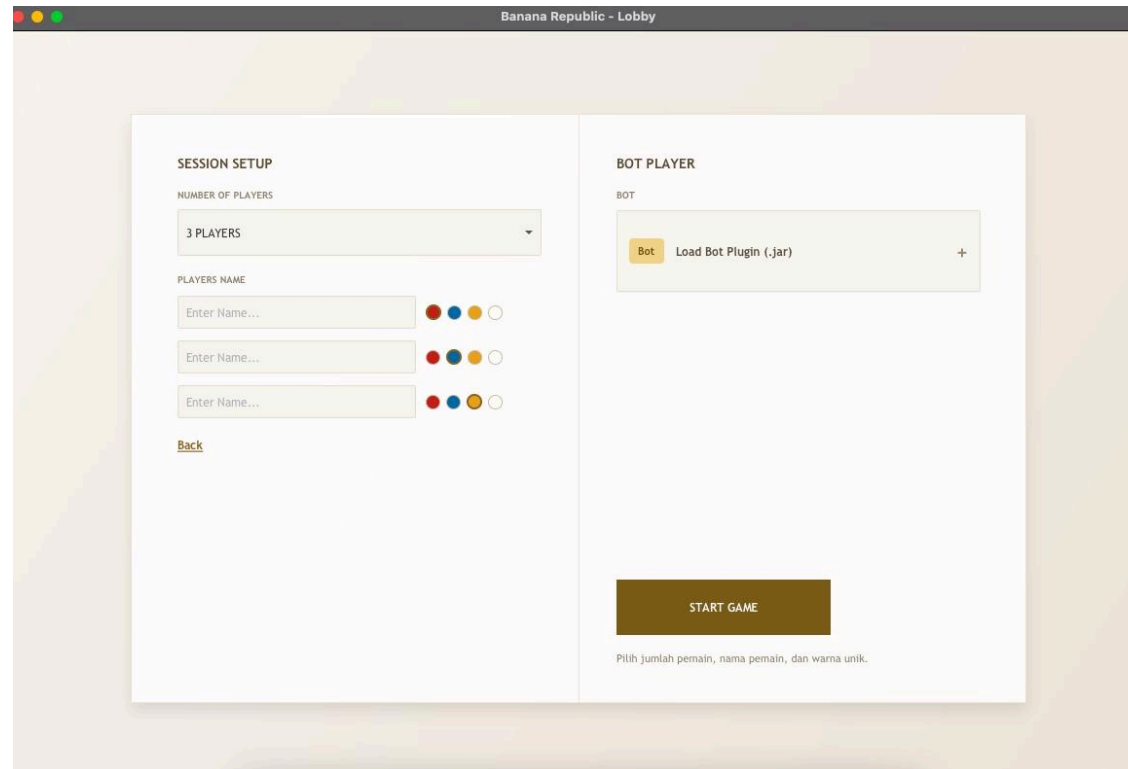
1. Deskripsi Umum Aplikasi

Banana Republic adalah aplikasi permainan papan strategi berbasis JavaFX yang mengadaptasi mekanisme Catan ke dalam tema kepulauan Nimon. Pemain bersaing mengumpulkan sumber daya, membangun Pipa Transportasi, Pos Pantau, dan Laboratorium, melakukan perdagangan, menggunakan Kartu Temuan Dr. Neroifa, serta menghindari gangguan Nimon Ungu untuk mencapai 10 Poin terlebih dahulu.

Aplikasi ini mendukung permainan 3-4 pemain dengan papan heksagonal, sistem giliran, produksi sumber daya berdasarkan dadu, perdagangan antar pemain dan pelabuhan, finite resource bank, save/load permainan, timer giliran, serta sistem plugin untuk menambahkan kartu temuan baru secara dinamis melalui file JAR. Implementasi dirancang secara modular dengan pemisahan antara model permainan, service, validator, event, plugin, persistence, dan antarmuka pengguna.



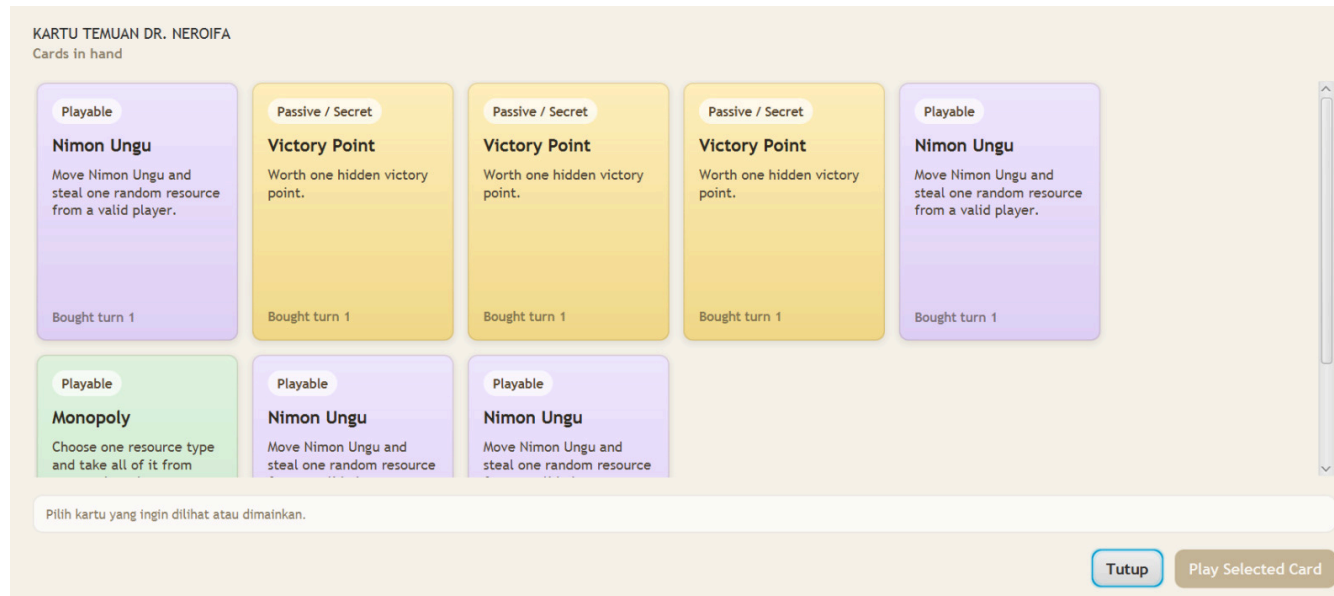
Gambar 1.1 Tampilan halaman login



Gambar 1.2 Tampilan halaman lobby



Gambar 1.3 Tampilan halaman permainan



Gambar 1.4 Tampilan halaman kartu temuan DR. Neorifa



Gambar 1.5 Tampilan pengaturan

2. Kakas GUI: <JavaFX Description>

Banana Republic dikembangkan menggunakan JavaFX sebagai kakas GUI. Pada aplikasi ini, JavaFX digunakan bersama FXML untuk memisahkan struktur antarmuka dari logika program, CSS untuk mengatur gaya visual, dan Maven untuk mengelola dependency seperti javafx-controls dan javafx-fxml. File FXML digunakan untuk mendefinisikan berbagai layar, seperti main menu, lobby, game, dialog trade, dialog

kartu, settings plugin, transisi giliran, dialog discard, dialog victory, dan hasil permainan. Sementara itu, logika tiap layar ditangani oleh controller Java yang terhubung melalui anotasi `@FXML`.

Lifecycle penggunaan JavaFX pada aplikasi ini dimulai dari pemanggilan `launch(args)` pada kelas `BananaRepublicApp`. Setelah JavaFX runtime aktif, method `start(Stage primaryStage)` dipanggil, lalu aplikasi melakukan inisialisasi melalui `AppInitializer`. Pada tahap ini, aplikasi membuat `ServiceRegistry`, `GameEventBus`, `PluginRegistry`, `CardService`, `ViewLoader`, `DialogManager`, dan `SceneManager`, kemudian membuka layar awal melalui `sceneManager.showMainMenu()`. Untuk menampilkan layar, `SceneManager` memuat file FXML menggunakan `ViewLoader/FXMLLoader`, membuat Scene dari root node FXML, menambahkan stylesheet `main.css`, mengatur properti Stage, lalu memanggil `stage.show()`. Setelah GUI tampil, interaksi pengguna diproses secara event-driven melalui handler seperti `onAction` pada FXML dan method controller yang diberi anotasi `@FXML`.

Komponen utama yang dibutuhkan agar GUI JavaFX dapat muncul adalah `Application`, `Stage`, `Scene`, dan `Node`. `BananaRepublicApp` berperan sebagai class turunan `Application` yang menjadi entry point JavaFX. Stage adalah window utama aplikasi yang diterima melalui parameter `start(Stage primaryStage)`. Scene adalah container untuk seluruh tampilan pada satu layar, sedangkan root node adalah elemen paling atas dari scene graph, misalnya `VBox`, `BorderPane`, atau `StackPane` yang didefinisikan dalam file FXML. Selain itu, aplikasi ini menggunakan pola FXML-Controller, yaitu FXML untuk mendeskripsikan layout dan controller untuk menangani event, binding data, navigasi scene, serta pemanggilan dialog.

JavaFX menyediakan berbagai komponen yang digunakan dalam aplikasi ini. Komponen layout yang digunakan antara lain `VBox`, `HBox`, `BorderPane`, `StackPane`, `Pane`, dan `Region` untuk mengatur struktur layar. Komponen kontrol yang digunakan meliputi `Button`, `Label`, `TextField`, `ComboBox`, `TextArea`, `TextInputDialog`, dan `Alert` untuk input, aksi, informasi, serta dialog. JavaFX juga menyediakan shape seperti `Circle`, `Polygon`, dan `Rectangle`, yang pada aplikasi ini dipakai untuk pilihan warna pemain, ikon tombol, bentuk tile, token angka, dan elemen papan permainan. Selain komponen bawaan JavaFX, aplikasi juga membuat custom component seperti `BoardView`, `HexTileView`, `DiceView`, `TimerView`, `ResourceCardView`, dan `PlayerPanelView` agar tampilan game board, dadu, timer, resource, dan status pemain lebih modular serta mudah digunakan ulang.

3. Plugin & Class Loader

Fitur plugin pada Banana Republic digunakan untuk memuat kartu eksperimen tambahan dari file eksternal berbentuk JAR. Prosesnya dimulai dari `SettingsPluginController`, ketika user memilih file JAR melalui `FileChooser`. File tersebut diberikan ke `ExperimentCardPluginLoader`, lalu

divalidasi oleh PluginValidator untuk memastikan file benar-benar .jar. Setelah itu, JarClassScanner membaca isi JAR menggunakan JarFile, mencari entry .class, lalu mengubah path class menjadi nama class Java penuh agar dapat dimuat saat runtime.

Class yang ditemukan dimuat menggunakan URLClassLoader, kemudian dicek apakah class tersebut mengimplementasikan ExperimentCard. Jika sesuai, plugin divalidasi lagi agar berupa concrete class, memiliki no-argument constructor, bertipe CardType.PLUGIN, memiliki id dengan prefix PLUGIN-, dan mengoverride applyEffect(GameState, Player). Setelah validasi berhasil, class plugin dibungkus menjadi ExperimentCardPlugin beserta PluginMetadata, lalu diregistrasikan ke PluginRegistry. Jika game sedang aktif, kartu plugin langsung dibuat dan dimasukkan ke DevelopmentDeck, sehingga kartu baru dapat dibeli dan dimainkan dalam permainan.

4. Class Diagram



Link [Class Diagram](#)

Class diagram program Banana Republic dibagi menjadi beberapa kelompok utama: app, core, service, event, listener, plugin, ui, validator, timer, dan data. Bagian core menjadi pusat model permainan, berisi objek domain seperti GameState, Board, AbstractPlayer, ResourceBank, DevelopmentDeck, AbstractBuilding, dan kartu eksperimen. Alur permainan dikendalikan oleh GameEngine, sedangkan aturan khusus seperti membangun, berdagang, melempar dadu, produksi resource, robber, dan kartu dipisahkan ke kelas-kelas service seperti BuildService, TradeService, DiceService, ResourceProductionService, RobberService, dan CardService.

Objek-objek dalam program berkomunikasi melalui dua pola. Pertama, GameController atau controller UI memanggil service melalui AppContext dan ServiceRegistry, lalu service memodifikasi GameState, Board, player, resource, atau kartu sesuai aksi pemain. Kedua, setelah terjadi perubahan penting, service menerbitkan event melalui GameEventBus, misalnya BuildingPlacedEvent, DiceRolledEvent, ResourceChangedEvent, TradeCompletedEvent, atau GameWonEvent. Event tersebut kemudian diterima oleh kelas-kelas listener seperti BoardChangeListener, ResourceChangeListener, TradeListener, TurnChangeListener, dan TimerListener untuk memperbarui tampilan atau menjalankan respons tambahan.

5. Konsep OOP

5.1. Inheritance

5.1.1. Player

Player adalah interface yang mendefinisikan kontrak dasar pemain (getName, addResource, removeResource). **AbstractPlayer** mengimplementasi interface tersebut dan menyediakan implementasi beserta field-field umum seperti inventory, cardHand, score, supply, dan stats. **HumanPlayer** adalah kelas yang mewarisi semua itu dari **AbstractPlayer**; ia tidak perlu mendefinisikan ulang method karena semuanya sudah tersedia.

(src/main/java/com/bananarepublic/core/player/HumanPlayer.java) merupakan anak dari kelas AbstractPlayer.

```
package com.bananarepublic.core.player;

import com.bananarepublic.core.resource.ResourceType;

import java.util.Objects;

public abstract class AbstractPlayer implements Player {
    private final PlayerId id;
    private final String name;
    private final PlayerColor color;
    private final PlayerInventory inventory;
    private final PlayerCardHand cardHand;
    private final PlayerScore score;
    private final PlayerSupply supply;
    private final PlayerStats stats;
    ...
}
```

```
package com.bananarepublic.core.player;

public final class HumanPlayer extends AbstractPlayer {
    public HumanPlayer(PlayerId id, String name, PlayerColor color) {
        super(id, name, color);
    }
}
```

5.1.2. Building

AbstractBuilding menyimpan atribut umum semua bangunan, ownerId, buildType, dan victoryPoints. Ketiga subclass (Settlement, Laboratory, Pipe) tidak perlu mendefinisikan ulang field tersebut, masing-masing hanya memanggil super() dengan nilai spesifikasinya.

(src/main/java/com/bananarepublic/core/building/Settlement.java),
 (src/main/java/com/bananarepublic/core/building/Laboratory.java),
 (src/main/java/com/bananarepublic/core/building/Pipe.java),
 merupakan anak dari kelas AbstractBuilding.

```
public abstract class AbstractBuilding {
    private final PlayerId ownerId;
    private final BuildType buildType;
    private final int victoryPoints;

    protected AbstractBuilding(PlayerId ownerId, BuildType buildType, int victoryPoints) {
        this.ownerId = Objects.requireNonNull(ownerId, "ownerId");
        this.buildType = Objects.requireNonNull(buildType, "buildType");
        this.victoryPoints = victoryPoints;
    }
    ...
}
```

```
package com.bananarepublic.core.building;

import com.bananarepublic.core.player.PlayerId;

public final class Settlement extends AbstractBuilding {
    public Settlement(PlayerId ownerId) {
        super(ownerId, BuildType.SETTLEMENT, 1);
    }
}
```

```
package com.bananarepublic.core.building;

import com.bananarepublic.core.player.PlayerId;

public final class Laboratory extends AbstractBuilding {
    public Laboratory(PlayerId ownerId) {
        super(ownerId, BuildType.LABORATORY, 2);
    }
}
```

```
package com.bananarepublic.core.building;

import com.bananarepublic.core.player.PlayerId;

public final class Pipe extends AbstractBuilding {
    public Pipe(PlayerId ownerId) {
        super(ownerId, BuildType.PIPE, 0);
    }
}
```

```

    }
}

```

5.1.3. Exception

GameException adalah root exception untuk aplikasi yang extends **RuntimeException**. **InvalidPlacementException** memperluas **GameException** untuk kategori kesalahan penempatan bangunan. **IntersectionOccupiedException** memperluas **InvalidPlacementException** dengan informasi **IntersectionId** yang bermasalah, ini membentuk hierarki tiga tingkat sehingga catch bisa dilakukan di level manapun.

Kelas **IntersectionOccupiedException** (src/main/java/com/bananarepublic/exception/build/IntersectionOccupiedException.java) merupakan anak dari kelas **InvalidPlacementException**. (src/main/java/com/bananarepublic/exception/build/InvalidPlacementException.java), kelas **InvalidPlacementException** merupakan anak dari kelas **GameException** (src/main/java/com/bananarepublic/exception/GameException.java).

```

package com.bananarepublic.exception;

public class GameException extends RuntimeException {
    public GameException(String message) {
        super(message);
    }

    public GameException(String message, Throwable cause) {
        super(message, cause);
    }
}

```

```
package com.bananarepublic.exception.build;

import com.bananarepublic.exception.GameException;

public class InvalidPlacementException extends GameException {
    public InvalidPlacementException(String message) {
        super(message);
    }
}
```

```
package com.bananarepublic.exception.build;

import com.bananarepublic.core.board.IntersectionId;

public class IntersectionOccupiedException extends InvalidPlacementException {
    private final IntersectionId intersectionId;

    public IntersectionOccupiedException(IntersectionId intersectionID) {
        super("Intersection " + intersectionID.value() + " is already occupied.");
        this.intersectionId = intersectionID;
    }

    public IntersectionId getIntersectionId() {
        return intersectionId;
    }
}
```

5.2. Composition

5.2.1. AbstractPlayer

(src/main/java/com/bananarepublic/core/player/AbstractPlayer.java) memiliki tepat satu PlayerInventory, satu PlayerCardHand, satu PlayerScore, satu PlayerSupply, dan satu PlayerStats. Objek-objek ini dibuat langsung di dalam konstruktor AbstractPlayer, sehingga setiap pemain selalu memiliki inventori, kartu, skor, stok bangunan, dan statistiknya sendiri.

```
package com.bananarepublic.core.player;

import com.bananarepublic.core.resource.ResourceType;

import java.util.Objects;

public abstract class AbstractPlayer implements Player {
    private final PlayerId id;
    private final String name;
    private final PlayerColor color;
    private final PlayerInventory inventory;
    private final PlayerCardHand cardHand;
    private final PlayerScore score;
    private final PlayerSupply supply;
    private final PlayerStats stats;

    protected AbstractPlayer(PlayerId id, String name, PlayerColor color) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Player name cannot be blank");
        }
        this.id = Objects.requireNonNull(id, "id");
    }
}
```

```
        this.name = name;
        this.color = Objects.requireNonNull(color, "color");
        this.inventory = new PlayerInventory();
        this.cardHand = new PlayerCardHand();
        this.score = new PlayerScore();
        this.supply = new PlayerSupply();
        this.stats = new PlayerStats();
    }

    public PlayerId getId() {
        return id;
    }

    @Override
    public String getName() {
        return name;
    }

    public PlayerColor getColor() {
        return color;
    }

    public PlayerInventory getInventory() {
        return inventory;
    }

    public PlayerCardHand getCardHand() {
        return cardHand;
    }

    public PlayerScore getScore() {
        return score;
    }
```

```

    }

    public PlayerSupply getSupply() {
        return supply;
    }

    public PlayerStats getStats() {
        return stats;
    }

    @Override
    public int getResourceCount(ResourceType type) {
        return inventory.getResourceCount(type);
    }

    @Override
    public void addResource(ResourceType type, int amount) {
        inventory.addResource(type, amount);
    }

    @Override
    public void removeResource(ResourceType type, int amount) {
        inventory.spendResource(type, amount);
    }
}

```

5.2.2. Board

Kelas Board (src/main/java/com/bananarepublic/core/board/Board.java) memiliki banyak HexTile, banyak Intersection, banyak Path, banyak Harbor, dan tepat satu Robber. Komposisi ini membentuk struktur papan permainan, karena board menjadi objek utama yang menyimpan seluruh elemen fisik permainan seperti petak heksagonal, persimpangan, jalur, pelabuhan, dan posisi Nimun Ungu.


```
package com.bananarepublic.core.board;

import java.util.List;
import java.util.Map;
import java.util.Optional;

public final class Board {
    private final Map<String, HexTile> hexTiles;
    private final Map<IntersectionId, Intersection> intersections;
    private final Map<PathId, Path> paths;
    private final List<Harbor> harbors;
    private final Robber robber;

    public Board(Map<String, HexTile> hexTiles, Map<IntersectionId, Intersection> intersections,
Map<PathId, Path> paths, List<Harbor> harbors, Robber robber) {
        this.hexTiles = Map.copyOf(hexTiles);
        this.intersections = Map.copyOf(intersections);
        this.paths = Map.copyOf(paths);
        this.harbors = List.copyOf(harbors);
        this.robber = robber;
    }

    public Map<String, HexTile> getHexTiles() {
        return hexTiles;
    }

    public Optional<HexTile> getHexTile(String id) {
        return Optional.ofNullable(hexTiles.get(id));
    }

    public Map<IntersectionId, Intersection> getIntersections() {
        return intersections;
    }

    public Map<PathId, Path> getPaths() {
        return paths;
    }
}
```

```

    }

    public List<Harbor> getHarbors() {
        return harbors;
    }

    public Robber getRobber() {
        return robber;
    }
}

```

5.2.3. PlayerCardHand

Kelas `PlayerCardHand` (`src/main/java/com/bananarepublic/core/player/PlayerCardHand.java`) memiliki banyak `OwnedExperimentCard` (`src/main/java/com/bananarepublic/core/card/OwnedExperimentCard.java`). Setiap kartu yang dibeli pemain akan dibungkus menjadi `OwnedExperimentCard` dan disimpan di dalam daftar kartu milik pemain.

```

public final class PlayerCardHand {
    private final List<OwnedExperimentCard> cards = new ArrayList<>();

    public OwnedExperimentCard addCard(ExperimentCard card, int boughtTurnNumber) {
        OwnedExperimentCard ownedCard = new OwnedExperimentCard(card, boughtTurnNumber);
        cards.add(ownedCard);
        return ownedCard;
    }

    public Optional<OwnedExperimentCard> findCard(ExperimentCardId cardId) {
        Objects.requireNonNull(cardId, "cardId");
        return cards.stream()
            .filter(card -> card.getCardId().equals(cardId))
            .findFirst();
    }
}

```

```

    public OwnedExperimentCard removeCard(ExperimentCardId cardId) {
        OwnedExperimentCard ownedCard = findCard(cardId)
            .orElseThrow(() -> new IllegalArgumentException("Card is not in hand: " +
cardId.value()));
        cards.remove(ownedCard);
        return ownedCard;
    }

    public int countByType(CardType type) {
        Objects.requireNonNull(type, "type");
        return (int) cards.stream()
            .filter(card -> card.getType() == type)
            .count();
    }

    public List<OwnedExperimentCard> getCards() {
        return List.copyOf(cards);
    }
}

```

5.3. Interface

5.3.1. GameEvent

Interface **GameEvent** (src/main/java/com/bananarepublic/event/GameEvent.java). Digunakan sebagai kontrak umum untuk seluruh event permainan. Interface ini memiliki method `occurredAt()` untuk mencatat waktu terjadinya event. Contoh implementasinya adalah **DiceRolledEvent**, **PhaseChangedEvent**, **GameWonEvent**, **ResourceProducedEvent**, dan **TradeCompletedEvent**.

```

package com.bananarepublic.event;

import java.time.Instant;

```

```
public interface GameEvent {  
    Instant occurredAt();  
}
```

```
package com.bananarepublic.event;  
  
import com.bananarepublic.core.dice.DiceRoll;  
import com.bananarepublic.core.player.PlayerId;  
  
import java.time.Instant;  
  
public record DiceRolledEvent(PlayerId playerId, DiceRoll diceRoll, Instant  
occurredAt) implements GameEvent {  
    public DiceRolledEvent(PlayerId playerId, DiceRoll diceRoll) {  
        this(playerId, diceRoll, Instant.now());  
    }  
}
```

```
package com.bananarepublic.event;  
  
import java.time.Instant;  
  
import com.bananarepublic.core.game.GamePhase;  
  
public record PhaseChangedEvent(GamePhase previousPhase, GamePhase currentPhase,  
Instant occurredAt) implements GameEvent {  
    public PhaseChangedEvent(GamePhase previousPhase, GamePhase currentPhase) {
```

```

        this(previousPhase, currentPhase, Instant.now());
    }
}

```

```

package com.bananarepublic.event;

import com.bananarepublic.core.player.PlayerId;

import java.time.Instant;

public record GameWonEvent(PlayerId winnerId, int victoryPoints, Instant occurredAt)
    implements GameEvent {
    public GameWonEvent(PlayerId winnerId, int victoryPoints) {
        this(winnerId, victoryPoints, Instant.now());
    }
}

```

5.3.2. GameEventListener

Interface **GameEventListener<T extends GameEvent>** (src/main/java/com/bananarepublic/event/GameEventListener.java) digunakan sebagai kontrak listener untuk menerima event dari sistem event bus. Interface ini memiliki method `onEvent(T event)`. Contoh implementasinya adalah **TurnChangeListener**, **TradeListener**, **TimerListener**, **ResourceChangeListener**, dan **GameResultListener**.

```

package com.bananarepublic.event;

@FunctionalInterface
public interface GameEventListener<T extends GameEvent> {

```

```
void onEvent(T event);
}
```

5.3.3. ExperimentCard

(src/main/java/com/bananarepublic/core/card/ExperimentCard.java) digunakan sebagai kontrak untuk semua kartu temuan dalam permainan. Interface ini mendefinisikan method getId(), getType(), getCardName(), dan getDescription(). Implementasinya diturunkan melalui **AbstractExperimentCard**, lalu digunakan oleh kartu seperti **KnightCard**, **MonopolyCard**, **RoadBuildingCard**, dan **VictoryPointCard**.

```
package com.bananarepublic.core.card;

import com.bananarepublic.core.game.GameState;
import com.bananarepublic.core.player.Player;

public interface ExperimentCard {
    default ExperimentCardId getId() {
        return new ExperimentCardId("PLUGIN-" + sanitizeId(getClass().getName()));
    }

    default CardType getType() {
        return CardType.PLUGIN;
    }

    String getCardName();

    String getDescription();

    void applyEffect(GameState state, Player player);
}
```

```
private static String sanitizeId(String raw) {  
    return raw.replaceAll("[^A-Za-z0-9._-]", "_");  
}  
}
```

```
package com.bananarepublic.core.card;  
  
import java.util.Objects;  
  
public abstract class AbstractExperimentCard implements ExperimentCard {  
    private final ExperimentCardId id;  
    private final CardType type;  
    private final String cardName;  
    private final String description;  
  
    protected AbstractExperimentCard(ExperimentCardId id, CardType type, String  
cardName, String description) {  
        if (cardName == null || cardName.isBlank()) {  
            throw new IllegalArgumentException("Card name cannot be blank");  
        }  
        if (description == null || description.isBlank()) {  
            throw new IllegalArgumentException("Card description cannot be blank");  
        }  
        this.id = Objects.requireNonNull(id, "id");  
        this.type = Objects.requireNonNull(type, "type");  
        this.cardName = cardName;  
        this.description = description;  
    }  
  
    @Override
```

```
public ExperimentCardId getId() {  
    return id;  
}  
  
@Override  
public CardType getType() {  
    return type;  
}  
  
@Override  
public String getCardName() {  
    return cardName;  
}  
  
@Override  
public String getDescription() {  
    return description;  
}  
}
```

5.3.4. BoardGenerator

(src/main/java/com/bananarepublic/core/board/generator/BoardGenerator.java) digunakan sebagai kontrak untuk kelas pembuat papan permainan. Interface ini memiliki method generate() yang menghasilkan **BoardGenerationResult**. Implementasi yang digunakan pada aplikasi adalah **StandardBoardGenerator**.

```
package com.bananarepublic.core.board.generator;  
  
public interface BoardGenerator {  
    BoardGenerationResult generate();  
}
```



```
}

```

```
public final class StandardBoardGenerator implements BoardGenerator {
    private static final double HEX_SIZE = 100.0;
    private static final TerrainType[] TERRAINS = {
        TerrainType.HUTAN, TerrainType.BUKIT, TerrainType.LADANG, TerrainType.GUNUNG,
        TerrainType.KEBUN_PISANG,
        TerrainType.HUTAN, TerrainType.LADANG, TerrainType.BUKIT, TerrainType.KEBUN_PISANG,
        TerrainType.GUNUNG,
        TerrainType.GURUN,
        TerrainType.HUTAN, TerrainType.LADANG, TerrainType.KEBUN_PISANG, TerrainType.BUKIT,
        TerrainType.GUNUNG,
        TerrainType.HUTAN, TerrainType.LADANG, TerrainType.KEBUN_PISANG
    };
    private static final int[] TOKENS = {5, 2, 6, 3, 8, 10, 9, 12, 11, 4, 0, 8, 10, 9, 4, 5, 6, 3,
    11};
    ...

```

5.3.5. DiceRoller

Interface **DiceRoller** (src/main/java/com/bananarepublic/core/dice/DiceRoller.java) digunakan sebagai kontrak untuk mekanisme pelemparan dadu. Interface ini memiliki method roll() yang menghasilkan **DiceRoll**. Implementasinya adalah **RandomDiceRoller** untuk lemparan acak dan **ManualDiceRoller** untuk lemparan manual.

```
package com.bananarepublic.core.dice;

public interface DiceRoller {

```

```
DiceRoll roll();  
}
```

```
package com.bananarepublic.core.dice;  
  
import java.util.Objects;  
import java.util.Random;  
  
public final class RandomDiceRoller implements DiceRoller {  
    private final Random random;  
  
    public RandomDiceRoller() {  
        this(new Random());  
    }  
  
    public RandomDiceRoller(Random random) {  
        this.random = Objects.requireNonNull(random, "random");  
    }  
  
    @Override  
    public DiceRoll roll() {  
        return DiceRoll.of(rollDie(), rollDie());  
    }  
  
    private int rollDie() {  
        return random.nextInt(6) + 1;  
    }  
}
```

```

package com.bananarepublic.core.dice;

public final class ManualDiceRoller implements DiceRoller {
    private DiceRoll nextRoll;

    public ManualDiceRoller(int firstValue, int secondValue) {
        setNextRoll(firstValue, secondValue);
    }

    public void setNextRoll(int firstValue, int secondValue) {
        this.nextRoll = DiceRoll.of(firstValue, secondValue);
    }

    @Override
    public DiceRoll roll() {
        return nextRoll;
    }
}

```

5.4. Method Overriding dan Method Overloading

5.4.1. Method Overriding roll() pada kelas RandomDiceRoller

(src/main/java/com/bananarepublic/core/dice/RandomDiceRoller.java) method ini meng-override method roll() dari interface **DiceRoller**.

```

package com.bananarepublic.core.dice;

```

```
import java.util.Objects;
import java.util.Random;

public final class RandomDiceRoller implements DiceRoller {
    private final Random random;

    public RandomDiceRoller() {
        this(new Random());
    }

    public RandomDiceRoller(Random random) {
        this.random = Objects.requireNonNull(random, "random");
    }

    @Override
    public DiceRoll roll() {
        return DiceRoll.of(rollDie(), rollDie());
    }

    private int rollDie() {
        return random.nextInt(6) + 1;
    }
}
```

5.4.2. Method Overriding roll() pada kelas ManualDiceRoller

(src/main/java/com/bananarepublic/core/dice/ManualDiceRoller.java) method ini meng-override method roll() dari interface DiceRoller, tetapi hasil dadunya berasal dari nilai manual.

```
package com.bananarepublic.core.dice;

public final class ManualDiceRoller implements DiceRoller {
    private DiceRoll nextRoll;

    public ManualDiceRoller(int firstValue, int secondValue) {
        setNextRoll(firstValue, secondValue);
    }

    public void setNextRoll(int firstValue, int secondValue) {
        this.nextRoll = DiceRoll.of(firstValue, secondValue);
    }

    @Override
    public DiceRoll roll() {
        return nextRoll;
    }
}
```

5.4.3. Method Overriding getName(), getResourceCount(), addResource(), dan removeResource() pada kelas AbstractPlayer

(src/main/java/com/bananarepublic/core/player/AbstractPlayer.java) method-method ini meng-override kontrak dari interface Player.

```
public abstract class AbstractPlayer implements Player {
    private final PlayerId id;
    private final String name;
```

```
private final PlayerColor color;
private final PlayerInventory inventory;
private final PlayerCardHand cardHand;
private final PlayerScore score;
private final PlayerSupply supply;
private final PlayerStats stats;

...

public PlayerId getId() {
    return id;
}

@Override
public String getName() {
    return name;
}

public PlayerColor getColor() {
    return color;
}

public PlayerInventory getInventory() {
    return inventory;
}

public PlayerCardHand getCardHand() {
    return cardHand;
}

public PlayerScore getScore() {
    return score;
}
```

```
public PlayerSupply getSupply() {  
    return supply;  
}  
  
public PlayerStats getStats() {  
    return stats;  
}  
  
@Override  
public int getResourceCount(ResourceType type) {  
    return inventory.getResourceCount(type);  
}  
  
@Override  
public void addResource(ResourceType type, int amount) {  
    inventory.addResource(type, amount);  
}  
  
@Override  
public void removeResource(ResourceType type, int amount) {  
    inventory.spendResource(type, amount);  
}  
}
```

5.4.4. Method Overriding takeTurn() pada kelas GreedyBotStrategy

(src/main/java/com/bananarepublic/core/bot/strategy/GreedyBotStrategy.java) meng-override method takeTurn() dari interface PlayerStrategy. Method ini menentukan aksi yang akan dilakukan bot saat gilirannya berjalan.

```
@Override
public Action takeTurn(GameState state) {
    if (state.getPhase() != GamePhase.PLAYER_ACTIONS) {
        return Action.noOp();
    }

    AbstractPlayer player = state.getCurrentPlayer();
    if (player.getInventory().hasResources(CardCostConfig.getExperimentCardCost())
        && state.getDevelopmentDeck().remainingCount() > 0) {
        return Action.buyExperimentCard();
    }

    for (ResourceType offeredType : ResourceType.values()) {
        if (player.getResourceCount(offeredType) < 4) {
            continue;
        }

        ResourceType requestedType = chooseNeededResource(player, offeredType);
        if (requestedType != null && state.getBank().hasResource(requestedType, 1)) {
            return Action.harborTrade(offeredType, requestedType);
        }
    }

    return Action.endTurn();
}
```


5.4.5. Method Overriding start() pada kelas BananaRepublicApp

(src/main/java/com/bananarepublic/BananaRepublicApp.java) meng-override method start() dari JavaFX Application.

```
public class BananaRepublicApp extends Application {  
    @Override  
    public void start(Stage primaryStage) {  
        new AppInitializer().initialize(primaryStage);  
    }  
  
    public static void main(String[] args) {  
        launch(args);  
    }  
}
```

5.5. Polymorphism

5.5.1. Method publish(GameEvent event) pada kelas GameEventBus

(src/main/java/com/bananarepublic/event/GameEventBus.java) menerima parameter bertipe interface **GameEvent**, tetapi method ini dipanggil dengan berbagai objek event konkret seperti **DiceRolledEvent**, **BuildingPlacedEvent**, **ResourceChangedEvent**, **TradeCompletedEvent**, dan **GameWonEvent**. Dengan demikian, event bus dapat memproses berbagai jenis event melalui satu tipe umum.

```
public final class GameEventBus {  
    private final Map<Class<? extends GameEvent>, List<GameEventListener<? extends  
GameEvent>>> listeners = new ConcurrentHashMap<>();
```

```

    public <T extends GameEvent> void register(Class<T> eventType,
GameEventListener<? super T> listener) {
        Objects.requireNonNull(eventType, "eventType");
        Objects.requireNonNull(listener, "listener");
        listeners.computeIfAbsent(eventType, ignored -> new ArrayList<>())
            .add(castListener(listener));
    }
    public void publish(GameEvent event) {
        Objects.requireNonNull(event, "event");
        listeners.forEach((registeredType, registeredListeners) -> {
            if (registeredType.isAssignableFrom(event.getClass())) {
                List.copyOf(registeredListeners).forEach(listener ->
notifyListener(listener, event));
            }
        });
    }
}

```

5.5.2. Kelas DevelopmentDeck

(src/main/java/com/bananarepublic/core/card/DevelopmentDeck.java) memiliki atribut **Deque<ExperimentCard>**, tetapi isi deck dibuat dari berbagai kelas seperti **KnightCard**, **RoadBuildingCard**, **MonopolyCard**, dan **VictoryPointCard** melalui **CardFactory** (src/main/java/com/bananarepublic/core/card/CardFactory.java). Semua kartu tersebut diperlakukan sebagai **ExperimentCard**.

```

public final class DevelopmentDeck {
    private final Deque<ExperimentCard> cards;

    public DevelopmentDeck(List<ExperimentCard> cards) {
        this.cards = new ArrayDeque<>(Objects.requireNonNull(cards, "cards"));
    }
}

```

```

    public static DevelopmentDeck standardDeck() {
        return standardDeck(new Random());
    }

    public static DevelopmentDeck standardDeck(Random random) {
        Objects.requireNonNull(random, "random");
        List<ExperimentCard> cards = new
ArrayList<>(CardFactory.createStandardCards());
        Collections.shuffle(cards, random);
        return new DevelopmentDeck(cards);
    }

    public ExperimentCard draw() {
        if (cards.isEmpty()) {
            throw new DeckEmptyException("Experiment card deck is empty");
        }
        return cards.removeFirst();
    }

```

```

    public static ExperimentCard create(CardType type, ExperimentCardId id) {
        return switch (type) {
            case KNIGHT -> new KnightCard(id);
            case ROAD_BUILDING -> new RoadBuildingCard(id);
            case MONOPOLY -> new MonopolyCard(id);
            case VICTORY_POINT -> new VictoryPointCard(id);
        };
    }

```

Kode di atas menunjukkan bahwa DevelopmentDeck hanya menyimpan kartu sebagai ExperimentCard, sedangkan objek konkretnya dapat berupa KnightCard, RoadBuildingCard, MonopolyCard, atau VictoryPointCard.

5.5.3. Kelas DiceService

(src/main/java/com/bananarepublic/service/DiceService.java) memiliki atribut bertipe interface **DiceRoller**. Method rollDice() cukup memanggil diceRoller.roll(), sementara implementasi dapat berupa **RandomDiceRoller** atau **ManualDiceRoller**. Dengan pola ini, cara pelemparan dadu dapat diganti tanpa mengubah logic utama pada DiceService.

```
public final class DiceService {
    private final DiceRoller diceRoller;
    private final GameEventBus eventBus;

    public DiceService(DiceRoller diceRoller, GameEventBus eventBus) {
        this.diceRoller = Objects.requireNonNull(diceRoller, "diceRoller");
        this.eventBus = Objects.requireNonNull(eventBus, "eventBus");
    }

    public DiceRoll rollDice(GameState state) {
        Objects.requireNonNull(state, "state");
        TurnValidator.requireStatus(state, GameStatus.IN_PROGRESS);
        TurnValidator.requirePhase(state, GamePhase.ROLL_DICE);

        DiceRoll diceRoll = diceRoller.roll();
        eventBus.publish(new DiceRolledEvent(state.getCurrentPlayer().getId(), diceRoll));
        return diceRoll;
    }
}
```

```
package com.bananarepublic.core.dice;

public interface DiceRoller {
    DiceRoll roll();
}
```

```
public final class ManualDiceRoller implements DiceRoller {

    ...

    @Override
    public DiceRoll roll() {
        return nextRoll;
    }

}
```

```
public final class RandomDiceRoller implements DiceRoller {

    ...

    @Override
    public DiceRoll roll() {
        return DiceRoll.of(rollDie(), rollDie());
    }

}
```

Kode tersebut menunjukkan bahwa DiceService tidak bergantung pada implementasi dadu tertentu. Selama sebuah kelas mengimplementasikan DiceRoller, kelas tersebut dapat digunakan oleh DiceService.

5.6. Java API Collection

5.6.1. Kelas GameState

Kelas GameState (src/main/java/com/bananarepublic/core/game/GameState.java) memiliki List<AbstractPlayer> untuk menyimpan seluruh pemain dalam permainan. Kelas ini juga memanfaatkan Stream API pada method findPlayerById() dengan players.stream().filter(...) untuk mencari pemain berdasarkan PlayerId.

```

public final class GameState {

    ...

    public GameState(List<AbstractPlayer> players, ResourceBank bank) {
        this(players, bank, DevelopmentDeck.standardDeck());
    }

    public GameState(List<AbstractPlayer> players, ResourceBank bank, DevelopmentDeck
developmentDeck) {
        GameRules.requireValidPlayerCount(players.size());
        this.players = new ArrayList<>(players);
        this.bank = Objects.requireNonNull(bank, "bank");
        this.developmentDeck = Objects.requireNonNull(developmentDeck, "developmentDeck");
        this.phase = GamePhase.DETERMINE_START_PLAYER;
        this.status = GameStatus.NOT_STARTED;
        this.currentPlayerIndex = 0;
        this.turnNumber = 0;
        this.initialForwardPlacementCount = 0;
        this.initialReversePlacementCount = 0;
        this.experimentCardPlayedThisTurn = false;
    }

    public static GameState newGame(List<AbstractPlayer> players) {
        return new GameState(players, new ResourceBank());
    }

    public List<AbstractPlayer> getPlayers() {
        return Collections.unmodifiableList(players);
    }

    public List<AbstractPlayer> getAllPlayers() {
        return getPlayers();
    }

    public AbstractPlayer getCurrentPlayer() {

```

```

        return players.get(currentPlayerIndex);
    }

    public Optional<AbstractPlayer> findPlayerById(PlayerId playerId) {
        Objects.requireNonNull(playerId, "playerId");
        return players.stream()
            .filter(player -> player.getId().equals(playerId))
            .findFirst();
    }

```

5.6.2. Kelas Board

Kelas Board (src/main/java/com/bananarepublic/core/board/Board.java) memiliki Map<String, HexTile>, Map<IntersectionId, Intersection>, Map<PathId, Path>, dan List<Harbor>. Struktur ini digunakan untuk menyimpan seluruh komponen papan permainan, seperti tile heksagonal, intersection, path, dan harbor.

```

public final class Board {
    private final Map<String, HexTile> hexTiles;
    private final Map<IntersectionId, Intersection> intersections;
    private final Map<PathId, Path> paths;
    private final List<Harbor> harbors;
    private final Robber robber;

    public Board(Map<String, HexTile> hexTiles, Map<IntersectionId, Intersection> intersections,
        Map<PathId, Path> paths, List<Harbor> harbors, Robber robber) {
        this.hexTiles = Map.copyOf(hexTiles);
        this.intersections = Map.copyOf(intersections);
        this.paths = Map.copyOf(paths);
        this.harbors = List.copyOf(harbors);
        this.robber = robber;
    }

    public Map<String, HexTile> getHexTiles() {

```

```

        return hexTiles;
    }

    public Optional<HexTile> getHexTile(String id) {
        return Optional.ofNullable(hexTiles.get(id));
    }

    public Map<IntersectionId, Intersection> getIntersections() {
        return intersections;
    }

    public Map<PathId, Path> getPaths() {
        return paths;
    }

    public List<Harbor> getHarbors() {
        return harbors;
    }

    public Robber getRobber() {
        return robber;
    }
}

```

5.6.3. Kelas DevelopmentDeck

Kelas `DevelopmentDeck` (`src/main/java/com/bananarepublic/core/card/DevelopmentDeck.java`) memiliki `Deque<ExperimentCard>` untuk merepresentasikan deck kartu temuan. Deque digunakan karena kartu diambil dari bagian depan deck melalui `removeFirst()` pada method `draw()`.

```

public final class DevelopmentDeck {
    private final Deque<ExperimentCard> cards;
}

```



```

public DevelopmentDeck(List<ExperimentCard> cards) {
    this.cards = new ArrayDeque<>(Objects.requireNonNull(cards, "cards"));
}

public static DevelopmentDeck standardDeck() {
    return standardDeck(new Random());
}

public static DevelopmentDeck standardDeck(Random random) {
    Objects.requireNonNull(random, "random");
    List<ExperimentCard> cards = new
ArrayList<>(CardFactory.createStandardCards());
    Collections.shuffle(cards, random);
    return new DevelopmentDeck(cards);
}

public ExperimentCard draw() {
    if (cards.isEmpty()) {
        throw new DeckEmptyException("Experiment card deck is empty");
    }
    return cards.removeFirst();
}

```

5.6.4. Kelas PlayerCardHand

Kelas `PlayerCardHand` (`src/main/java/com/bananarepublic/core/player/PlayerCardHand.java`) memiliki `List<OwnedExperimentCard>` untuk menyimpan kartu yang dimiliki pemain. Kelas ini juga memakai Stream API pada method `findCard()` dan `countByType()` untuk mencari kartu tertentu dan menghitung jumlah kartu berdasarkan tipe.

```

public final class PlayerCardHand {
    private final List<OwnedExperimentCard> cards = new ArrayList<>();

    public OwnedExperimentCard addCard(ExperimentCard card, int boughtTurnNumber) {
        OwnedExperimentCard ownedCard = new OwnedExperimentCard(card, boughtTurnNumber);
        cards.add(ownedCard);
        return ownedCard;
    }

    public Optional<OwnedExperimentCard> findCard(ExperimentCardId cardId) {
        Objects.requireNonNull(cardId, "cardId");
        return cards.stream()
            .filter(card -> card.getCardId().equals(cardId))
            .findFirst();
    }

    public OwnedExperimentCard removeCard(ExperimentCardId cardId) {
        OwnedExperimentCard ownedCard = findCard(cardId)
            .orElseThrow(() -> new IllegalArgumentException("Card is not in hand: " +
cardId.value()));
        cards.remove(ownedCard);
        return ownedCard;
    }

    public int countByType(CardType type) {
        Objects.requireNonNull(type, "type");
        return (int) cards.stream()
            .filter(card -> card.getType() == type)
            .count();
    }

    public List<OwnedExperimentCard> getCards() {
        return List.copyOf(cards);
    }
}

```

5.7. SOLID

5.7.1. Single Responsibility Principle

1. Kelas **BuildService** (src/main/java/com/bananarepublic/service/BuildService.java) bertugas menangani aksi pembangunan seperti membangun pipa, pos pantau, dan laboratorium. Validasi aturan pembangunan dipisahkan ke **BuildValidator** (src/main/java/com/bananarepublic/validator/BuildValidator.java).

```
public final class BuildValidator {
    public void validateBuildPhase(GameState state) {
        Objects.requireNonNull(state, "state");
        if (state.getStatus() != GameStatus.IN_PROGRESS) {
            throw new InvalidBuildException("Build is only allowed while the game is in
progress");
        }
        if (state.getPhase() != GamePhase.PLAYER_ACTIONS) {
            throw new InvalidBuildException("Build is only allowed during player actions");
        }
    }

    public void validateCanAfford(AbstractPlayer player, ResourceBundle cost) {
//Hanya validasi
    }

    public void validateHasSupply(AbstractPlayer player, BuildType buildType) {
//Hanya validasi
    }
}
```

```

        public void validatePipe(Board board, PathId pathId, PlayerId playerId) {
//Hanya validasi
        }

        public void validateSettlement(Board board, IntersectionId intersectionId, PlayerId playerId)
        {
//Hanya validasi
        }

        public void validateLaboratoryUpgrade(Board board, IntersectionId intersectionId, PlayerId
playerId) {
//Hanya Validasi

        public void validateInitialSettlement(Board board, IntersectionId intersectionId) {
//Hanya validasi

        public void validateInitialPipe(Board board, PathId pathId, IntersectionId
settlementIntersectionId) {
//Hanya validasi
        }

```

2. Kelas **DiceService** (src/main/java/com/bananarepublic/service/DiceService.java) hanya bertugas menangani proses pelemparan dadu dan publikasi event hasil dadu. Validasi fase permainan tetap didelegasikan ke TurnValidator

```

public final class DiceService {
    private final DiceRoller diceRoller;
    private final GameEventBus eventBus;

    public DiceService(DiceRoller diceRoller, GameEventBus eventBus) {
        this.diceRoller = Objects.requireNonNull(diceRoller, "diceRoller");
        this.eventBus = Objects.requireNonNull(eventBus, "eventBus");
    }

```

```

    public DiceRoll rollDice(GameState state) {
        Objects.requireNonNull(state, "state");
        TurnValidator.requireStatus(state, GameStatus.IN_PROGRESS);
        TurnValidator.requirePhase(state, GamePhase.ROLL_DICE);

        DiceRoll diceRoll = diceRoller.roll();
        eventBus.publish(new DiceRolledEvent(state.getCurrentPlayer().getId(),
diceRoll));
        return diceRoll;
    }
}

```

3. Kelas **SceneManager** (src/main/java/com/bananarepublic/ui/router/SceneManager.java) hanya bertanggung jawab mengatur perpindahan scene JavaFX. Proses pemuatan FXML dipisahkan ke ViewLoader, sedangkan dialog dipisahkan ke **DialogManager**.

```

public final class SceneManager {
    private static final int DEFAULT_WIDTH = 1180;
    private static final int DEFAULT_HEIGHT = 760;

    private final Stage stage;
    private final ViewLoader viewLoader;
    private final DialogManager dialogManager;

    public SceneManager(Stage stage, ViewLoader viewLoader, DialogManager
dialogManager) {
        this.stage = Objects.requireNonNull(stage, "stage");
        this.viewLoader = Objects.requireNonNull(viewLoader, "viewLoader");
        this.dialogManager = Objects.requireNonNull(dialogManager, "dialogManager");
    }
}

```

```
}

public void showMainMenu() {
    show("/fxml/main-menu.fxml", "Banana Republic");
}

public void showLobby() {
    show("/fxml/lobby.fxml", "Banana Republic - Lobby");
}

public void showGame() {
    show("/fxml/game.fxml", "Banana Republic - Game");
}

public void showTurnTransition() {
    show("/fxml/turn-transition.fxml", "Banana Republic - Turn Transition");
}

public void showGameResult() {
    show("/fxml/game-result.fxml", "Banana Republic - Result");
}

public DialogManager dialogManager() {
    return dialogManager;
}

private void show(String fxmlPath, String title) {
    Parent root = viewLoader.load(fxmlPath);
    Scene scene = new Scene(root, DEFAULT_WIDTH, DEFAULT_HEIGHT);
    scene.getStylesheets().add(Objects.requireNonNull(
        getClass().getResource("/css/main.css"),
        "Stylesheet not found: /css/main.css"
    ));
}
```

```

        ).toExternalForm());

        stage.setTitle(title);
        stage.setMinWidth(800);
        stage.setMinHeight(540);
        stage.setScene(scene);
        stage.show();
    }
}

```

5.7.2. *Open/closed Principle*

1. Interface **DiceRoller** (src/main/java/com/bananarepublic/core/dice/DiceRoller.java) memungkinkan penambahan cara pelemparan dadu baru tanpa mengubah DiceService. Saat ini implementasinya adalah **RandomDiceRoller** dan **ManualDiceRoller**.

```

public interface DiceRoller {
    DiceRoll roll();
}

```

2. Interface **ExperimentCard** (src/main/java/com/bananarepublic/core/card/ExperimentCard.java) memungkinkan penambahan jenis kartu baru tanpa mengubah kelas yang menyimpan kartu sebagai tipe umum, seperti **DevelopmentDeck**.

```

public interface ExperimentCard {
    default ExperimentCardId getId() {
        return new ExperimentCardId("PLUGIN-" + sanitizeId(getClass().getName()));
    }
}

```

```

    default CardType getType() {
        return CardType.PLUGIN;
    }

    String getCardName();

    String getDescription();

    void applyEffect(GameState state, Player player);

    private static String sanitizeId(String raw) {
        return raw.replaceAll("[^A-Za-z0-9._-]", "_");
    }
}

```

3. Interface **BoardGenerator** (src/main/java/com/bananarepublic/core/board/generator/BoardGenerator.java) memungkinkan penambahan generator board baru tanpa mengubah kode yang bergantung pada kontrak generate(). Implementasi saat ini adalah **StandardBoardGenerator**.

```

public interface BoardGenerator {
    BoardGenerationResult generate();
}

```

5.7.3. Liskov Substitution Principle

1. **RandomDiceRoller** dan **ManualDiceRoller** (src/main/java/com/bananarepublic/core/dice/) dapat digunakan sebagai pengganti **DiceRoller** karena keduanya menyediakan method **roll()** yang menghasilkan DiceRoll.

```

public final class ManualDiceRoller implements DiceRoller {

```



```

private DiceRoll nextRoll;

public ManualDiceRoller(int firstValue, int secondValue) {
    setNextRoll(firstValue, secondValue);
}

public void setNextRoll(int firstValue, int secondValue) {
    this.nextRoll = DiceRoll.of(firstValue, secondValue);
}

@Override
public DiceRoll roll() {
    return nextRoll;
}
}

```

```

public final class RandomDiceRoller implements DiceRoller {
    private final Random random;

    public RandomDiceRoller() {
        this(new Random());
    }

    public RandomDiceRoller(Random random) {
        this.random = Objects.requireNonNull(random, "random");
    }

    @Override
    public DiceRoll roll() {
        return DiceRoll.of(rollDie(), rollDie());
    }
}

```

```

    }

    private int rollDie() {
        return random.nextInt(6) + 1;
    }
}

```

2. **KnightCard**, **MonopolyCard**, **RoadBuildingCard**, dan **VictoryPointCard** dapat diperlakukan sebagai **ExperimentCard** melalui parent **AbstractExperimentCard**. Contohnya, **DevelopmentDeck** menyimpan semuanya sebagai **Deque<ExperimentCard>**.

```

public final class KnightCard extends AbstractExperimentCard {
    public KnightCard(ExperimentCardId id) {
        super(id, CardType.KNIGHT, "Nimon Ungu", "Move Nimon Ungu and steal one random resource from a valid player.");
    }
}

public final class MonopolyCard extends AbstractExperimentCard {
    public MonopolyCard(ExperimentCardId id) {
        super(id, CardType.MONOPOLY, "Monopoly", "Choose one resource type and take all of it from every other player.");
    }
}

public final class VictoryPointCard extends AbstractExperimentCard {
    public VictoryPointCard(ExperimentCardId id) {
        super(id, CardType.VICTORY_POINT, "Victory Point", "Worth one hidden victory point.");
    }
}

```

3. **Pipe**, **Settlement**, dan **Laboratory** dapat diperlakukan sebagai **AbstractBuilding**. Contohnya, **BuildingFactory** (src/main/java/com/bananarepublic/core/building/BuildingFactory.java) mengembalikan **AbstractBuilding**, tetapi objek dapat berbeda sesuai **BuildType**.

```
public final class Pipe extends AbstractBuilding {
    public Pipe(PlayerId ownerId) {
        super(ownerId, BuildType.PIPE, 0);
    }
}

public final class Settlement extends AbstractBuilding {
    public Settlement(PlayerId ownerId) {
        super(ownerId, BuildType.SETTLEMENT, 1);
    }
}

public final class Laboratory extends AbstractBuilding {
    public Laboratory(PlayerId ownerId) {
        super(ownerId, BuildType.LABORATORY, 2);
    }
}
```

```
public final class BuildingFactory {
    private BuildingFactory() {
    }

    public static AbstractBuilding create(BuildType buildType, PlayerId ownerId) {
        return switch (buildType) {
            case PIPE -> new Pipe(ownerId);
            case SETTLEMENT -> new Settlement(ownerId);
        };
    }
}
```

```

        case LABORATORY -> new Laboratory(ownerId);
    };
}

```

5.7.4. Interface Segregation Principle

1. Interface **DiceRoller** hanya memiliki method **roll()**, sehingga kelas pelempar dadu tidak dipaksa mengimplementasikan method lain yang tidak dibutuhkan.

```

public interface DiceRoller {
    DiceRoll roll();
}

```

2. Interface **BoardGenerator** hanya memiliki method **generate()**, sehingga kelas generator papan hanya perlu fokus pada proses pembuatan board.

```

public interface BoardGenerator {
    BoardGenerationResult generate();
}

```

3. Interface **GameEventListener<T extends GameEvent>** hanya memiliki method **onEvent(T event)**, sehingga listener event cukup mengimplementasikan cara merespons event tanpa kewajiban method tambahan.

```

@FunctionalInterface
public interface GameEventListener<T extends GameEvent> {
    void onEvent(T event);
}

```

```
}
```

5.7.5. Dependency Inversion Principle

1. Kelas **DiceService** bergantung pada abstraksi **DiceRoller**, bukan langsung pada **RandomDiceRoller** atau **ManualDiceRoller**. Implementasi dadu diberikan melalui konstruktor.

```
public final class DiceService {
    private final DiceRoller diceRoller;
    private final GameEventBus eventBus;

    public DiceService(DiceRoller diceRoller, GameEventBus eventBus) {
        this.diceRoller = Objects.requireNonNull(diceRoller, "diceRoller");
        this.eventBus = Objects.requireNonNull(eventBus, "eventBus");
    }

    public DiceRoll rollDice(GameState state) {
        Objects.requireNonNull(state, "state");
        TurnValidator.requireStatus(state, GameStatus.IN_PROGRESS);
        TurnValidator.requirePhase(state, GamePhase.ROLL_DICE);

        DiceRoll diceRoll = diceRoller.roll();
        eventBus.publish(new DiceRolledEvent(state.getCurrentPlayer().getId(),
diceRoll));
        return diceRoll;
    }
}
```

2. Kelas **GameEventBus** (src/main/java/com/bananarepublic/event/GameEventBus.java) bekerja dengan abstraksi **GameEvent** dan **GameEventListener**, sehingga event bus tidak perlu bergantung pada kelas event atau listener konkret tertentu.

```
public final class GameEventBus {
    private final Map<Class<? extends GameEvent>, List<GameEventListener<? extends GameEvent>>>
listeners = new ConcurrentHashMap<>();

    public <T extends GameEvent> void register(Class<T> eventType, GameEventListener<? super T>
listener) {
        Objects.requireNonNull(eventType, "eventType");
        Objects.requireNonNull(listener, "listener");
        listeners.computeIfAbsent(eventType, ignored -> new ArrayList<>())
            .add(castListener(listener));
    }
    public void publish(GameEvent event) {
        Objects.requireNonNull(event, "event");
        listeners.forEach((registeredType, registeredListeners) -> {
            if (registeredType.isAssignableFrom(event.getClass())) {
                List.copyOf(registeredListeners).forEach(listener -> notifyListener(listener,
event));
            }
        });
    }

    public int listenerCount(Class<? extends GameEvent> eventType) {
        return listeners.getDefault(eventType, List.of()).size();
    }

    @SuppressWarnings("unchecked")
    private <T extends GameEvent> GameEventListener<? extends GameEvent>
castListener(GameEventListener<? super T> listener) {
        return (GameEventListener<? extends GameEvent>) listener;
    }
    @SuppressWarnings("unchecked")
```

```

    private <T extends GameEvent> void notifyListener(GameEventListener<T> listener, GameEvent
event) {
        listener.onEvent((T) event);
    }
}

```

5.8. Design Pattern

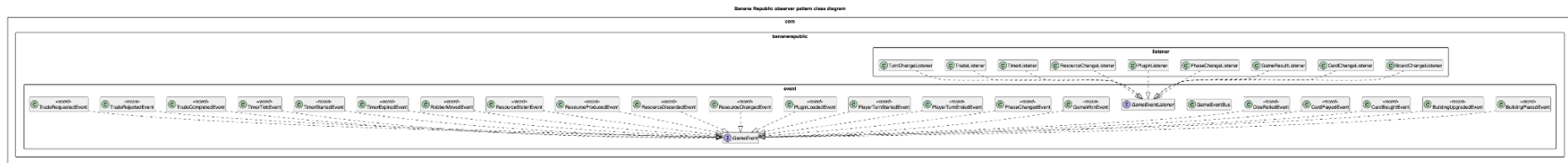
5.8.1. Observer Pattern

Observer Pattern diimplementasikan pada **GameEventBus** (src/main/java/com/bananarepublic/event/GameEventBus.java) sebagai kelas publisher/event dispatcher yang mengelola dan mendistribusikan event ke listener yang terdaftar. Tujuannya adalah memisahkan komponen yang menghasilkan event, seperti **TurnManager**, **BuildService**, **CardService**, dan **ResourceProductionService**, dari komponen yang merespons event, seperti **TurnChangeListener**, **BoardChangeListener**, **ResourceChangeListener**, **TradeListener**, dan listener lain yang mengimplementasikan **GameEventListener**. **GameEventBus** menyimpan map dari tipe event ke daftar listener, lalu ketika event dipublish melalui method `publish()`, semua listener yang terdaftar untuk event tersebut akan dinotify melalui method `onEvent()`.

Kelebihannya adalah loose coupling antara komponen penghasil event dan komponen penerima event. Misalnya, **TurnManager** cukup memanggil `eventBus.publish(new PlayerTurnEndedEvent(...))` tanpa perlu mengetahui listener mana saja yang akan merespons event tersebut. Listener seperti **TurnChangeListener** dapat menangani **PlayerTurnStartedEvent** dan **PlayerTurnEndedEvent** melalui interface **GameEventListener**, sehingga mekanisme respons terhadap perubahan game dapat ditambahkan tanpa mengubah kode publisher.

Jika tidak diterapkan, setiap service yang menghasilkan event harus memiliki direct reference ke semua komponen yang perlu dinotify ketika suatu aksi terjadi. Misalnya, **TurnManager** harus mengetahui dan memanggil langsung komponen yang menangani perubahan giliran, **BuildService** harus memanggil langsung komponen yang memperbarui board, dan

ResourceProductionService harus memanggil langsung komponen yang memperbarui resource. Hal ini menyebabkan tight coupling, membuat kode lebih sulit dikembangkan, dan berpotensi menimbulkan circular dependencies.



Link gambar [class diagram observer pattern](#)

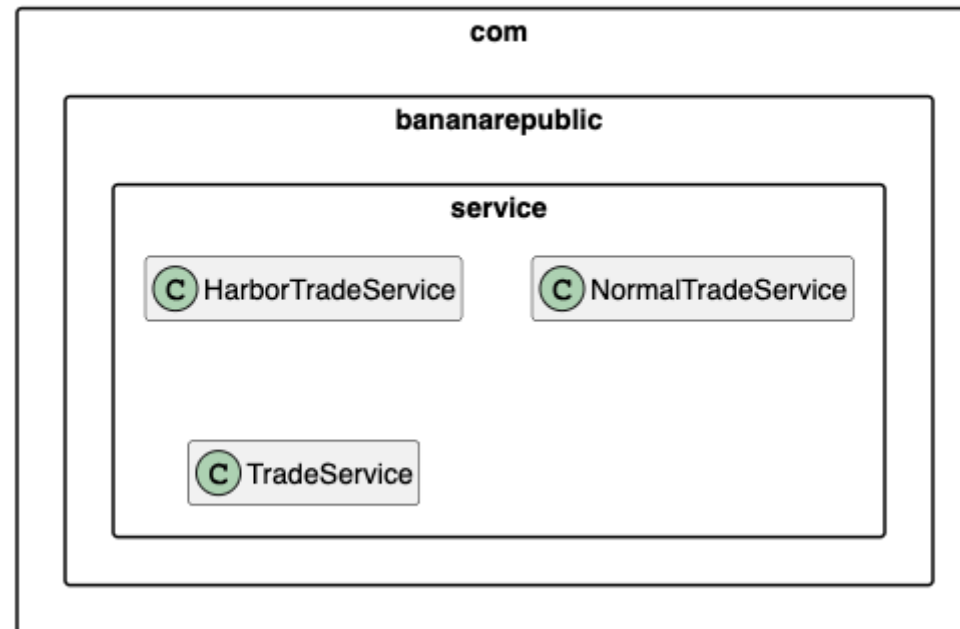
5.8.2. Facade Pattern

Facade Pattern diterapkan pada **TradeService** (src/main/java/com/bananarepublic/service/TradeService.java) sebagai kelas facade yang menyederhanakan akses terhadap beberapa jenis layanan trade, yaitu **NormalTradeService** (src/main/java/com/bananarepublic/service/NormalTradeService.java) dan **HarborTradeService** (src/main/java/com/bananarepublic/service/HarborTradeService.java). Tujuannya adalah menyediakan satu pintu akses untuk operasi trade tanpa membuat client code berinteraksi langsung dengan banyak service berbeda. **NormalTradeService** menangani trade antar pemain seperti request, accept, reject, counter-offer, dan pending trade, sedangkan **HarborTradeService** menangani trade dengan bank/harbor berdasarkan rasio harbor yang dimiliki pemain.

Kelebihannya adalah client code cukup menggunakan **TradeService** untuk menjalankan operasi trade, sehingga detail implementasi normal trade dan harbor trade tersembunyi di balik satu interface layanan yang lebih sederhana. Dengan pemisahan ini, logic trade antar pemain dan trade harbor tetap berada di kelas masing-masing, tetapi akses dari luar tetap terpusat melalui TradeService.

Jika tidak diterapkan, controller atau service lain harus mengetahui dan memanggil langsung **NormalTradeService** dan **HarborTradeService** sesuai jenis trade yang dilakukan. Akibatnya, logic pemilihan dan pemanggilan trade akan tersebar di banyak tempat, membuat kode lebih sulit dirawat.

Banana Republic facade pattern class diagram



5.9. Reflection

5.9.1. PluginValidator.java - Validasi Metode Plugin

(src/main/java/com/bananarepublic/plugin/PluginValidator.java) Method `validatePolymorphicEffect(Class<?>)` menggunakan reflection dengan `pluginClass.getMethod("applyEffect", GameState.class, Player.class)` untuk mendapatkan metode `applyEffect` dari kelas plugin secara dinamis. Kemudian menggunakan `method.getDeclaringClass()` untuk memverifikasi bahwa plugin card telah mengoverride metode tersebut (bukan menggunakan implementasi default dari `AbstractExperimentCard`). Ini memastikan bahwa setiap plugin card mengimplementasikan efek custom mereka sendiri.

```
private void validatePolymorphicEffect(Class<?> pluginClass) {
    try {
        Method method = pluginClass.getMethod("applyEffect", GameState.class, Player.class);
        if (method.getDeclaringClass().equals(AbstractExperimentCard.class)) {
            throw new PluginValidationException("Plugin card must override
applyEffect(GameState, Player): " + pluginClass.getName());
        }
    } catch (NoSuchMethodException e) {
        throw new PluginValidationException("Plugin card must define applyEffect(GameState,
Player): " + pluginClass.getName(), e);
    }
}
```

5.9.2. ExperimentCardPluginLoader.java - Dynamic Class Loading untuk Plugin

(src/main/java/com/bananarepublic/plugin/card/ExperimentCardPluginLoader.java) Method `loadClass(ClassLoader, String)` menggunakan `Class.forName(className, false, classLoader)` untuk melakukan dynamic class loading dari plugin JAR secara runtime. Parameter `false` mengindikasikan bahwa class tidak langsung di-initialize, hanya di-load. Custom `ClassLoader` digunakan untuk memastikan class dari JAR file dapat di-load dengan benar dalam konteks plugin system.

```
private Class<?> loadClass(ClassLoader classLoader, String className) {
    try {
        return Class.forName(className, false, classLoader);
    } catch (ClassNotFoundException | LinkageError e) {
        throw new PluginLoadException("Unable to load plugin class: " + className, e);
    }
}
```

5.9.3. ExperimentCardPlugin.java - Factory untuk Plugin Card Instance

(src/main/java/com/bananarepublic/plugin/card/ExperimentCardPlugin.java) Method createCard() menggunakan reflection mechanism yang sama dengan loader untuk cardClass.getDeclaredConstructor() untuk mendapatkan constructor dari stored class reference constructor.setAccessible(true) untuk mengatur akses untuk private constructor.

```
public ExperimentCard createCard() {
    try {
        Constructor<? extends ExperimentCard> constructor = cardClass.getDeclaredConstructor();
        constructor.setAccessible(true);
        return constructor.newInstance();
    } catch (InstantiationException | IllegalAccessException | InvocationTargetException |
NoSuchMethodException e) {
        throw new PluginLoadException("Unable to instantiate plugin card: " + metadata.className(), e);
    }
}
```

5.10. Multithreading

5.10.1. Kelas TimerTaskRunner

(src/main/java/com/bananarepublic/timer/TimerTaskRunner.java) menggunakan ScheduledExecutorService dengan Executors.newSingleThreadScheduledExecutor(...). Thread bernama banana-republic-turn-timer dibuat sebagai daemon thread untuk menjalankan timer giliran secara periodik.

```
public final class TimerTaskRunner {
    private final ScheduledExecutorService executorService;

    // create thread
    public TimerTaskRunner() {
        this.executorService = Executors.newSingleThreadScheduledExecutor(task -> {
            Thread thread = new Thread(task, "banana-republic-turn-timer");
            thread.setDaemon(true);
            return thread;
        });
    }
}
```

5.10.2. Method scheduleAtFixedRate() pada TimerTaskRunner

(src/main/java/com/bananarepublic/timer/TimerTaskRunner.java) menjadwalkan task berulang dengan interval tertentu. Fitur ini digunakan agar countdown timer dapat berjalan setiap detik tanpa menghentikan alur utama permainan.

```
public ScheduledFuture<?> scheduleAtFixedRate(Runnable task, long initialDelay, long period,
TimeUnit unit) {
    Objects.requireNonNull(task, "task");
    Objects.requireNonNull(unit, "unit");
    return executorService.scheduleAtFixedRate(task, initialDelay, period, unit);
}
```

```

    public void shutdown() {
        executorService.shutdownNow();
    }
}

```

5.10.3. Kelas CountdownTimer

(src/main/java/com/bananarepublic/timer/CountdownTimer.java) menggunakan `ScheduledFuture<?>` untuk menyimpan task timer yang sedang berjalan. Method `start()` menjadwalkan pemanggilan `tick()` setiap detik, sedangkan `stop()` membatalkan task tersebut.

```

public final class CountdownTimer {
    private final int durationSeconds;
    private final TimerTaskRunner taskRunner;
    private final IntConsumer tickHandler;
    private final Runnable expiredHandler;
    private final Object lock = new Object();
    private ScheduledFuture<?> scheduledTask;
    private int remainingSeconds;
    private TimerState state = TimerState.READY;

    public CountdownTimer(int durationSeconds, TimerTaskRunner taskRunner, IntConsumer
tickHandler, Runnable expiredHandler) {
        if (durationSeconds <= 0) {
            throw new IllegalArgumentException("Timer duration must be positive");
        }
        this.durationSeconds = durationSeconds;
        this.taskRunner = Objects.requireNonNull(taskRunner, "taskRunner");
        this.tickHandler = Objects.requireNonNull(tickHandler, "tickHandler");
        this.expiredHandler = Objects.requireNonNull(expiredHandler, "expiredHandler");
        this.remainingSeconds = durationSeconds;
    }
}

```

```
// calling runnable
public void start() {
    synchronized (lock) {
        cancelScheduledTask();
        remainingSeconds = durationSeconds;
        state = TimerState.RUNNING;
        scheduledTask = taskRunner.scheduleAtFixedRate(this::tick, 1, 1, TimeUnit.SECONDS);
    }
}

public void stop() {
    synchronized (lock) {
        if (state == TimerState.RUNNING) {
            state = TimerState.STOPPED;
        }
        cancelScheduledTask();
    }
}

public int getRemainingSeconds() {
    synchronized (lock) {
        return remainingSeconds;
    }
}

public TimerState getState() {
    synchronized (lock) {
        return state;
    }
}

private void tick() {
    int secondsLeft;
    boolean expired;

    synchronized (lock) {
```

```
        if (state != TimerState.RUNNING) {
            return;
        }

        remainingSeconds--;
        if (remainingSeconds <= 0) {
            remainingSeconds = 0;
            state = TimerState.EXPIRED;
            cancelScheduledTask();
            expired = true;
        } else {
            expired = false;
        }
        secondsLeft = remainingSeconds;
    }

    tickHandler.accept(secondsLeft);
    if (expired) {
        expiredHandler.run();
    }
}

private void cancelScheduledTask() {
    if (scheduledTask != null) {
        scheduledTask.cancel(false);
        scheduledTask = null;
    }
}
}
```

5.10.4. Kelas CountdownTimer

(src/main/java/com/bananarepublic/timer/CountdownTimer.java) menggunakan `synchronized` (lock) untuk menjaga konsistensi state timer seperti `remainingSeconds`, `state`, dan `scheduledTask`. Sinkronisasi ini diperlukan karena state timer dapat diakses dari thread berbeda.

```
// calling runnable
public void start() {
    synchronized (lock) {
...
    }
}

public void stop() {
    synchronized (lock) {
        if (state == TimerState.RUNNING) {
            state = TimerState.STOPPED;
        }
        cancelScheduledTask();
    }
}

public int getRemainingSeconds() {
    synchronized (lock) {
        return remainingSeconds;
    }
}

public TimerState getState() {
    synchronized (lock) {
        return state;
    }
}

private void tick() {
```



```

        int secondsLeft;
        boolean expired;

        synchronized (lock) {
            if (state != TimerState.RUNNING) {
                return;
            }
        }

        ...
    }
}

```

5.10.5. Kelas TurnTimer

(src/main/java/com/bananarepublic/timer/TurnTimer.java) menggunakan method `synchronized` pada `start()`, `stop()`, `getRemainingSeconds()`, `getState()`, dan `shutdown()`. Hal ini memastikan operasi timer giliran aman ketika diakses secara bersamaan.

```

public synchronized void start(PlayerId playerId, int turnNumber, Runnable onExpired) {
    Objects.requireNonNull(playerId, "playerId");
    Objects.requireNonNull(onExpired, "onExpired");
    stop();

    countdownTimer = new CountdownTimer(
        durationSeconds,
        taskRunner,
        remainingSeconds -> eventBus.publish(new TimerTickEvent(playerId, turnNumber,
remainingSeconds)),
        () -> {
            eventBus.publish(new TimerExpiredEvent(playerId, turnNumber));
            onExpired.run();
        }
    );
}

```

```
        eventBus.publish(new TimerStartedEvent(playerId, turnNumber, durationSeconds));
        countdownTimer.start();
    }

    public synchronized void stop() {
        if (countdownTimer != null) {
            countdownTimer.stop();
        }
    }

    public synchronized int getRemainingSeconds() {
        if (countdownTimer == null) {
            return 0;
        }
        return countdownTimer.getRemainingSeconds();
    }

    public synchronized TimerState getState() {
        if (countdownTimer == null) {
            return TimerState.READY;
        }
        return countdownTimer.getState();
    }

    public synchronized void shutdown() {
        stop();
        taskRunner.shutdown();
    }
}
```

5.11. Assertion

5.11.1. ResourceBank

(src/main/java/com/bananarepublic/core/resource/ResourceBank.java) menggunakan assert untuk memverifikasi resource count tidak pernah menjadi negatif setelah memberikan resource kepada player

Dan (src/main/java/com/bananarepublic/core/resource/ResourceBank.java) menggunakan assert untuk memverifikasi resource count tidak melebihi kapasitas maksimal bank

```
public void giveResource(ResourceType type, int amount) {
    resources.remove(type, amount);
    assert resources.get(type) >= 0 : "Resource bank count cannot be negative after giving " +
type;
}

public void receiveResource(ResourceType type, int amount) {
    resources.add(type, amount);
    assert resources.get(type) <= ResourceConfig.INITIAL_BANK_RESOURCE_COUNT
        : "Resource bank count cannot exceed finite bank size for " + type;
}
```

Post-condition assertion 1 yang menjamin invariant bahwa bank tidak bisa memiliki resource count negatif. Jika terjadi, akan throw AssertionError yang mengindikasikan bug dalam logic remove resource.

Post-condition assertion 2 yang memastikan bank memiliki kapasitas finite (bounded resource pool). Melanggar invariant ini mengindikasikan bug dalam logic distribusi atau penerimaan resource dari players.

5.11.2. TurnManager

(src/main/java/com/bananarepublic/core/game/TurnManager.java) menggunakan dua independent assertions untuk memvalidasi state transition ketika turn berakhir:

```
public final class TurnManager {
    private final GameEventBus eventBus;

    ...

    // called at the end of each player's turn
    public void endTurn(GameState state) {
        Objects.requireNonNull(state, "state");
        TurnValidator.requireCanEndTurn(state);

        eventBus.publish(new PlayerTurnEndedEvent(state.getCurrentPlayer().getId(),
state.getTurnNumber()));

        int previousPlayerIndex = state.getCurrentPlayerIndex();
        int previousTurnNumber = state.getTurnNumber();
        state.setCurrentPlayerIndex(nextPlayerIndex(state));

        state.setTurnNumber(state.getTurnNumber() + 1); //increment turn number
        state.resetExperimentCardPlayedThisTurn();
        changePhase(state, GamePhase.ROLL_DICE);

        assert state.getCurrentPlayerIndex() == (previousPlayerIndex + 1) % state.getPlayers().size()
            : "End turn must advance to the next player";
        assert state.getTurnNumber() == previousTurnNumber + 1 : "End turn must increment turn number by
one";
        publishTurnStarted(state);
    }
}
```

- Assertion 1 memverifikasi player index berubah dengan benar menggunakan modulo arithmetic (circular turn order mengelilingi semua player).
- Assertion 2 memverifikasi turn number bertambah tepat satu.
- Keduanya menjaga game loop mechanics dengan strict invariant checking.

5.11.3. DevelopmentDeck

(src/main/java/com/bananarepublic/core/card/DevelopmentDeck.java) menggunakan assert untuk memverifikasi bahwa proses mengambil kartu dari deck benar-benar mengurangi ukuran deck sebanyak satu dan kartu yang diambil tidak null.

```
public ExperimentCard draw() {
    if (cards.isEmpty()) {
        throw new DeckEmptyException("Experiment card deck is empty");
    }
    int previousSize = cards.size();
    ExperimentCard drawnCard = cards.removeFirst();
    assert cards.size() == previousSize - 1 : "Drawing a card must reduce deck size by one";
    assert drawnCard != null : "Development deck cannot draw a null card";
    return drawnCard;
}
```

Assertion ini berperan sebagai post-condition untuk operasi draw card. Jika ukuran deck tidak berkurang satu atau kartu yang diambil bernilai null, maka ada bug pada struktur data deck atau proses manipulasi kartu.

5.11.4. GameState

(src/main/java/com/bananarepublic/core/game/GameState.java) menggunakan assert untuk memastikan phase game tidak null setelah assignment.

```
public void setPhase(GamePhase phase) {
    this.phase = Objects.requireNonNull(phase, "phase");
    assert this.phase != null : "Game phase cannot be null after assignment";
}
```

Assertion ini menjaga invariant bahwa GameState selalu berada pada fase yang valid. Jika phase null, alur permainan seperti roll dice, player actions, robber, atau game over tidak dapat ditentukan dengan benar.

5.11.5. BuildService

(src/main/java/com/bananarepublic/service/BuildService.java)menggunakan assert untuk memastikan setelah settlement dibangun, intersection benar-benar berisi settlement milik current player.

```
intersection.placeBuilding(settlement);

assert intersection.getBuilding()
    .map(building -> building.getOwnerId().equals(player.getId())
        && building.getBuildType() == BuildType.SETTLEMENT)
    .orElse(false) : "Settlement build must place the current player's settlement";
```

Assertion ini merupakan post-condition dari proses buildSettlement(). Setelah validasi, pembayaran cost, pengurangan supply, dan penempatan building dilakukan, intersection harus berisi SETTLEMENT milik pemain yang sedang aktif. Jika assertion gagal, berarti ada bug pada proses placement building atau kepemilikan building.

6. Bonus Yang dikerjakan

6.1. Bonus Animasi Background

Pendekatan yang digunakan pada animasi background adalah dengan membuat animasi langsung pada area board, tepatnya di bagian lautan yang mengelilingi pulau. Implementasinya dibuat ringan dengan memisahkan animasi air ke komponen khusus **AnimatedTiledWaterView**, lalu komponen ini dipasang sebagai layer paling bawah pada **BoardView**. Animasi air tidak menggunakan banyak node JavaFX, melainkan hanya satu Canvas yang menggambar ulang pola tile kecil dari beberapa frame gambar. Dengan cara ini, rendering tetap ringan karena JavaFX tidak perlu mengelola banyak elemen visual yang bergerak satu per satu.

Selain animasi air, board juga menambahkan elemen ambience berupa awan transparan di pinggiran board menggunakan ImageView dengan file GIF. Awan dibuat mouseTransparent, sehingga tidak mengganggu interaksi pemain dengan tile, intersection, path, atau Nimon Ungu. Setiap kali board di-redraw atau ukuran board berubah, animasi air lama dihentikan melalui stopActiveWaterView() sebelum dibuat ulang. Hal ini penting agar tidak ada Timeline lama yang terus berjalan di background dan membebani UI Thread. Secara umum, animasi dibuat sebagai dekorasi visual pasif yang tidak ikut masuk ke logic game, sehingga tidak mengganggu mekanisme permainan.

```
public final class AnimatedTiledWaterView extends Pane {
    private static final List<String> FRAME_PATHS = List.of(
        "/images/animation/water/water-frame-0.png",
        "/images/animation/water/water-frame-1.png",
        "/images/animation/water/water-frame-2.png",
        "/images/animation/water/water-frame-3.png",
        "/images/animation/water/water-frame-4.png"
    );

    private static final double DEFAULT_TILE_SIZE = 64.0;
    private static final Duration FRAME_DURATION = Duration.millis(1000);

    private final Canvas canvas = new Canvas();
    private final List<Image> frames;
    private final Timeline timeline;
    private int frameIndex;
    private double tileSize = DEFAULT_TILE_SIZE;

    public AnimatedTiledWaterView(double width, double height) {
        this.frames = FRAME_PATHS.stream()
            .map(ResourceImageCache::get)
            .toList();

        getStyleClass().add("animated-tiled-water");
        setMouseTransparent(true);
        canvas.setMouseTransparent(true);
        getChildren().add(canvas);
    }
}
```

```

        setPrefSize(width, height);
        resizeCanvas(width, height);

        widthProperty().addListener((observable, oldValue, newValue) -> redraw());
        heightProperty().addListener((observable, oldValue, newValue) -> redraw());

        this.timeline = new Timeline(new KeyFrame(FRAME_DURATION, event -> nextFrame()));
        this.timeline.setCycleCount(Timeline.INDEFINITE);
        this.timeline.play();
        redraw();
    }

    public void stop() {
        timeline.stop();
    }

    public void setTileSize(double tileSize) {
        if (tileSize <= 0) {
            throw new IllegalArgumentException("tileSize must be positive");
        }
        this.tileSize = tileSize;
        redraw();
    }

    @Override
    protected void layoutChildren() {
        super.layoutChildren();
        resizeCanvas(getWidth(), getHeight());
    }

    private void nextFrame() {
        frameIndex = (frameIndex + 1) % frames.size();
        redraw();
    }

    private void resizeCanvas(double width, double height) {

```



```

        canvas.setWidth(Math.max(1, width));
        canvas.setHeight(Math.max(1, height));
        redraw();
    }

    private void redraw() {
        if (frames.isEmpty()) {
            return;
        }
        GraphicsContext gc = canvas.getGraphicsContext2D();
        gc.setImageSmoothing(false);
        double width = canvas.getWidth();
        double height = canvas.getHeight();
        gc.clearRect(0, 0, width, height);

        Image frame = frames.get(frameIndex);
        for (double y = 0; y < height; y += tileSize) {
            for (double x = 0; x < width; x += tileSize) {
                gc.drawImage(frame, x, y, tileSize, tileSize);
            }
        }
    }
}

```

6.2. Bonus Pemain Bot

Untuk bonus Pemain Bot, pendekatan yang digunakan adalah membuat sistem bot berbasis Strategy Pattern dan plugin loader. Program menyediakan interface `PlayerStrategy` sebagai kontrak umum untuk semua bot. Setiap bot wajib mengimplementasikan method `takeTurn(GameState state)` yang mengembalikan `Action`, misalnya `BUY_EXPERIMENT_CARD`, `HARBOR_TRADE`,

END_TURN, atau NO_OP. Dengan pendekatan ini, logic pengambilan keputusan bot dipisahkan dari game engine. Game engine hanya menjalankan action yang diberikan, sedangkan strategi bot dapat diganti-ganti tanpa mengubah logic permainan utama.

Bot diload dari file .jar pada saat pemain masih berada di lobby. BotPluginLoader membaca isi JAR, mencari class yang mengimplementasikan PlayerStrategy, lalu melakukan validasi menggunakan reflection, seperti memastikan class bersifat konkret, memiliki constructor tanpa parameter, dan memiliki metadata strategy yang valid. Setelah plugin bot berhasil diload, LobbyController menyimpannya ke PluginRegistry. Ketika permainan dimulai, slot pemain terakhir diganti menjadi BotPlayer yang menyimpan strategy hasil plugin tersebut. Pembatasan ini membuat plugin bot hanya bisa dimuat sebelum game dimulai, sehingga state permainan yang sedang berjalan tidak berubah secara tiba-tiba.

Saat giliran bot tiba, GameController mendeteksi bahwa current player adalah BotPlayer, lalu menjalankan BotTurnService. Service ini menangani bagian otomatis seperti roll dice, discard resource jika terkena dadu 7, memindahkan Nimun Ungu, lalu memanggil strategy.takeTurn(state) untuk menentukan aksi bot pada fase player action. Action yang dikembalikan kemudian dieksekusi oleh GameEngine. Dengan mekanisme ini, bot dapat berjalan otomatis, tetapi tetap mengikuti aturan validasi game yang sama seperti pemain manusia.

```
public interface PlayerStrategy {
    default String getStrategyId() {
        return "BOT-" + getClass().getName().replaceAll("[^A-Za-z0-9._-]", "_");
    }

    default String getStrategyName() {
        return getClass().getSimpleName();
    }

    default String getDescription() {
        return "A Banana Republic bot strategy.";
    }

    Action takeTurn(GameState state);
}
```

```
// BotPluginLoader.java
@Override
public List<BotPlugin> load(Path jarPath) {
    validator.validateJarPath(jarPath);
    URLClassLoader classLoader = createClassLoader(jarPath);
    List<BotPlugin> plugins = new ArrayList<>();

    for (String className : classScanner.scan(jarPath)) {
        Class<?> loadedClass = loadClass(classLoader, className);
        if (!PlayerStrategy.class.isAssignableFrom(loadedClass)) {
            continue;
        }

        validator.validatePlayerStrategyClass(loadedClass);
        Class<? extends PlayerStrategy> strategyClass =
            loadedClass.asSubclass(PlayerStrategy.class);

        PlayerStrategy prototype = instantiate(strategyClass);
        validator.validatePlayerStrategyInstance(prototype);

        plugins.add(toPlugin(jarPath, strategyClass, prototype));
    }

    if (plugins.isEmpty()) {
        throw new PluginValidationException(
            "No PlayerStrategy implementation found in plugin JAR: " + jarPath
        );
    }
    return List.copyOf(plugins);
}
```

```
// BotTurnService.java
private boolean executeStrategyActions(GameEngine engine, PlayerStrategy strategy, List<String> messages) {
    GameState state = engine.getState();
    for (int i = 0; i < MAX_ACTIONS_PER_TURN; i++) {
        Action action = strategy.takeTurn(state);
        if (action == null || action.getType() == com.bananarepublic.core.bot.ActionType.NO_OP) {
            messages.add(state.getCurrentPlayer().getName() + " (bot) tidak memilih aksi lanjutan.");
            return endTurn(engine, messages);
        }

        try {
            switch (action.getType()) {
                case END_TURN -> {
                    return endTurn(engine, messages);
                }
                case BUY_EXPERIMENT_CARD -> {
                    engine.buyExperimentCard();
                    messages.add(state.getCurrentPlayer().getName() + " (bot) membeli Kartu Temuan Dr. Neroifa.");
                }
                case HARBOR_TRADE -> {
                    engine.tradeWithHarbor(action.getOfferedResourceType(), action.getRequestedResourceType());
                    messages.add(state.getCurrentPlayer().getName() + " (bot) trade "
                        + action.getOfferedResourceType()
                        + " ke "
                        + action.getRequestedResourceType()
                        + " lewat Terminal Dagang Gro.");
                }
                case NO_OP -> {
                    return endTurn(engine, messages);
                }
            }
        } catch (RuntimeException exception) {
            messages.add(state.getCurrentPlayer().getName()
                + " (bot) gagal menjalankan aksi "
                + action.getType()
                + ": "
                + exception.getMessage());
            return endTurn(engine, messages);
        }
    }
}
```

```
        if (state.getStatus() == com.bananarepublic.core.game.GameStatus.GAME_OVER) {  
            return false;  
        }  
    }  
  
    messages.add(state.getCurrentPlayer().getName() + " (bot) mencapai batas aksi otomatis.");  
    return endTurn(engine, messages);  
}
```

6.3. Bonus SFX & BGM

Untuk bonus SFX (Sound Effects), pendekatan yang digunakan adalah membuat pemutar suara ringan khusus untuk efek pendek melalui kelas `SoundEffectPlayer`. Kelas ini memakai `AudioClip` dari `JavaFX`, bukan `MediaPlayer`, karena `AudioClip` memang cocok untuk suara singkat seperti klik mouse, lempar dadu, atau chime kemenangan. `AudioClip.play()` bersifat non-blocking dan dapat dipanggil berkali-kali tanpa menghentikan suara sebelumnya, sehingga beberapa SFX bisa overlap dengan SFX lain ketika aksi terjadi berdekatan. Agar tidak perlu memuat file audio berulang kali, setiap clip disimpan dalam `ConcurrentHashMap` sebagai cache berdasarkan path resource.

Implementasi SFX dibuat terpisah dari logic game utama. `Controller` hanya memanggil method seperti `playDiceRoll()` saat pemain melempar dadu, sedangkan `SceneManager` memasang event filter global untuk memainkan suara klik setiap kali pengguna menekan mouse kiri pada scene. Dengan cara ini, detail pemuatan file audio, volume, cache, dan error handling dikumpulkan di satu tempat. Jika file suara hilang atau perangkat audio bermasalah, exception ditangkap agar efek suara tidak pernah mengganggu jalannya permainan.

Untuk BGM, program memakai `MediaPlayer`. Kelas `BackgroundMusicPlayer` menyimpan playlist berisi beberapa file musik (`bgm-1.mp3`, `bgm-2.mp3`, dan `bgm-3.mp3`), lalu memainkan track secara berurutan. Ketika satu track selesai, callback `setOnEndOfMedia()` akan memanggil track berikutnya, dan index track dikembalikan ke awal menggunakan modulo sehingga musik dapat terus berulang selama permainan berlangsung. BGM hanya dimulai saat game aktif melalui `GameController`, lalu dihentikan ketika kembali ke main menu, lobby, game result, atau saat victory dialog muncul. Volume BGM juga dibuat sebagai `DoubleProperty`, sehingga dapat diatur dari settings melalui slider.

SFX

```
public final class SoundEffectPlayer {
    private static final SoundEffectPlayer INSTANCE = new SoundEffectPlayer();

    private static final String MOUSE_CLICK = "/sounds/mouse-click.wav";
    private static final String DICE_ROLL = "/sounds/dice-roll.wav";
    private static final String VICTORY_CHIME = "/sounds/victory-chime.wav";

    private final Map<String, AudioClip> clips = new ConcurrentHashMap<>();
    private volatile boolean enabled = true;

    private SoundEffectPlayer() {
    }

    public static SoundEffectPlayer getInstance() {
        return INSTANCE;
    }

    public void setEnabled(boolean enabled) {
        this.enabled = enabled;
    }

    public boolean isEnabled() {
        return enabled;
    }

    public void playMouseClicked() {
        play(MOUSE_CLICK, 0.28);
    }

    public void playDiceRoll() {
        play(DICE_ROLL, 0.60);
    }

    public void playVictoryChime() {
```

```

        play(VICTORY_CHIME, 0.75);
    }

    private void play(String resourcePath, double volume) {
        if (!enabled) {
            return;
        }

        try {
            AudioClip clip = clips.computeIfAbsent(resourcePath, this::loadClip);
            clip.setVolume(clamp(volume));
            clip.play();
        } catch (RuntimeException ignored) {
            // Sound effects must never prevent the game from running.
        }
    }

    private AudioClip loadClip(String resourcePath) {
        URL resource = SoundEffectPlayer.class.getResource(resourcePath);
        Objects.requireNonNull(resource, "Sound effect not found: " + resourcePath);
        return new AudioClip(resource.toExternalForm());
    }

    private double clamp(double volume) {
        return Math.max(0.0, Math.min(1.0, volume));
    }
}

```

BGM

```

public final class BackgroundMusicPlayer {
    private static final BackgroundMusicPlayer INSTANCE = new BackgroundMusicPlayer();
}

```

```
private static final String[] PLAYLIST = {
    "/sounds/bgm-1.mp3",
    "/sounds/bgm-2.mp3",
    "/sounds/bgm-3.mp3"
};
private static final double DEFAULT_VOLUME = 0.45;

private final DoubleProperty volume = new SimpleDoubleProperty(DEFAULT_VOLUME);

private MediaPlayer currentPlayer;
private int trackIndex;
private int consecutiveFailures;
private boolean playing;

private BackgroundMusicPlayer() {
    volume.addListener((observable, oldValue, newValue) -> applyVolume());
}

public static BackgroundMusicPlayer getInstance() {
    return INSTANCE;
}

public DoubleProperty volumeProperty() {
    return volume;
}

public double getVolume() {
    return volume.get();
}

public void setVolume(double value) {
    runOnFxThread(() -> volume.set(clamp(value)));
}

public void start() {
    runOnFxThread(() -> {
        if (playing) {
            applyVolume();
        }
    });
}
```



```
        return;
    }
    playing = true;
    playFrom(trackIndex, PLAYLIST.length);
});
}

public void stop() {
    runOnUiThread(() -> {
        playing = false;
        trackIndex = 0;
        consecutiveFailures = 0;
        disposeCurrentPlayer();
    });
}

private void playNextTrack() {
    if (!playing) {
        return;
    }
    trackIndex = (trackIndex + 1) % PLAYLIST.length;
    playFrom(trackIndex, PLAYLIST.length);
}

private void playFrom(int startIndex, int attemptsRemaining) {
    if (!playing) {
        return;
    }
    if (attemptsRemaining <= 0) {
        playing = false;
        consecutiveFailures = 0;
        return;
    }

    disposeCurrentPlayer();
    trackIndex = Math.floorMod(startIndex, PLAYLIST.length);

    try {
        currentPlayer = createPlayer(PLAYLIST[trackIndex]);
    }
}
```

```

        currentPlayer.setVolume(clamp(volume.get()));
        currentPlayer.setOnPlaying(() -> consecutiveFailures = 0);
        currentPlayer.setOnEndOfMedia(() -> {
            consecutiveFailures = 0;
            playNextTrack();
        });
        currentPlayer.setOnError(this::handleTrackError);
        currentPlayer.play();
    } catch (RuntimeException ignored) {
        trackIndex = (trackIndex + 1) % PLAYLIST.length;
        playFrom(trackIndex, attemptsRemaining - 1);
    }
}

private void handleTrackError() {
    if (!playing) {
        return;
    }
    consecutiveFailures++;
    if (consecutiveFailures >= PLAYLIST.length) {
        playing = false;
        consecutiveFailures = 0;
        disposeCurrentPlayer();
        return;
    }
    playNextTrack();
}

private MediaPlayer createPlayer(String resourcePath) {
    URL resource = BackgroundMusicPlayer.class.getResource(resourcePath);
    Objects.requireNonNull(resource, "Background music not found: " + resourcePath);
    return new MediaPlayer(new Media(resource.toExternalForm()));
}

private void applyVolume() {
    if (currentPlayer != null) {
        currentPlayer.setVolume(clamp(volume.get()));
    }
}

```

```

private void disposeCurrentPlayer() {
    if (currentPlayer == null) {
        return;
    }
    currentPlayer.stop();
    currentPlayer.dispose();
    currentPlayer = null;
}

private void runOnFxThread(Runnable task) {
    if (Platform.isFxApplicationThread()) {
        task.run();
    } else {
        Platform.runLater(task);
    }
}

private double clamp(double value) {
    return Math.max(0.0, Math.min(1.0, value));
}
}

```

7. Pembagian Tugas

- 13524122 / Nathaniel Christian
 - Classes, Services, Listeners, Events, Exceptions, Validators → Game: engine, phase, state, etc.; Dice; Card; Board & Building: intersection, point; Trade; Turn System; Resource Production; Player: stats; Timer; Scoring; Robber/Nimon Ungu;
 - UI (some design)
- 13524133 / M. Daffa Arrizki Yanma
 - Initial setup (sama Azri) dan exception
 - Solo carry laporan

- 13524135 / Varistha Devi
 - UI → Viewer, Controller, etc.
 - Bonus SFX, Animation
- 13524139 / Azri Arzaq Pohan
 - Core gameplay → Board, GameState, Player, Resource, Building; Initial setup; Build system & validation; GameEngine integration; Scoring/Victory; Robber/Nimon Ungu; Experiment cards & card effects; Plugin system → card plugin JAR, bot plugin JAR;
 - Bonus Bot Player; sebagian UI controller dan event bridge; Bonus BGM.
- 13524142 / Rasyad Satyatma
 - Class diagram
 - Save/Load game